

QA — Por qué necesitamos un rol humano dedicado

Autor: David Cabezas (dev full-stack junior)
Destinatario: José (decisión de hiring / scope QA)
Fecha: 2026-04-23
Contexto: Plataforma AMS (Mageam) — pivot reliability + integración SAP PM, cliente Goldfields.

TL;DR

Durante esta semana desarrollé con Claude (IA) más de **40 cambios** basados en feedback de Jorge. Claude **cubre las partes del QA que son código + flujo predecible**, pero **deja 5 clases de verificación que requieren una persona**. Sin un QA humano, esos cinco huecos llegan al cliente como bugs en producción.

Qué hace Claude (IA) bien — cubre ~60% del QA clásico

Área	Claude hace
Code review estático	Encuentra TODO, funciones sin uso, imports rotos, tipos mal escritos.
Test unitario	Genera pytest para endpoints backend, valida contratos entrada/salida.
Búsqueda de patrones peligrosos	Inyección HTML sin escape, SQL con interpolación, comparación insegura de secretos.
Verificar endpoints HTTP	curl contra /health, status codes, headers esperados.
Consistencia de copy/labels	Grep global para renombrar "Cancel WR" a "Cancelar Aviso" en todo el repo.
Refactor seguro	Extraer funciones, renombrar variables, mover lógica entre archivos.
Smoke test de build	docker build, npm run build, pytest suite completa.

Qué Claude NO puede hacer — 5 huecos que obligan a un QA humano

1. Testing exploratorio real en el navegador

- **Limitación IA:** No veo la pantalla. No puedo hacer click, arrastrar, completar wizards de varios pasos con datos sucios, probar en diferente tamaño de pantalla, ver si algo "se siente raro".
- **Ejemplo concreto esta semana:** Jorge clickó el botón "Cancel WR" tres veces y "no hacía nada". Yo validé que el endpoint funcionaba, el frontend llamaba correcto, el modal aparecía. **El bug real era que el botón no se renderizaba para status OT_CREADA** — algo que sólo se ve clickeando el aviso específico que él abrió. Me tomó 3 iteraciones con Jorge antes de entenderlo.

- **Qué hace un QA humano:** recorre flujos completos, prueba datos borde, combina permisos de rol distintos, reproduce el bug del usuario paso a paso.

2. Validación con datos reales del cliente (Goldfields)

- **Limitación IA:** Trabajo con `seed_demo_data.py` — datos sintéticos perfectos. El cliente sube su propio CSV con equipos reales, nomenclatura SAP específica, campos vacíos, encodings UTF-8 raros.
- **Ejemplo concreto esta semana:** Jorge vio duplicados "multímetro, limpiadora, destornilladores". Yo asumí que era bug de render — era dato sucio del seed pero también podría pasar con el data real.
- **Qué hace un QA humano:** importa el dataset de Goldfields, detecta validaciones faltantes, prueba los edge cases que **sólo existen en producción**.

3. Regresión visual — ¿se rompió algo que antes funcionaba?

- **Limitación IA:** Hice 40+ cambios hoy. No puedo ir clickeando las 30+ pantallas después de cada cambio para ver si algo que antes funcionaba ahora está roto (color, layout, botones fuera de lugar, modales que no cierran).
- **Ejemplo concreto esta semana:** Cambié "Cancel WR" a "Cancelar Aviso" y el botón Pantalla Completa del modal WR se mantuvo duplicado porque había **dos copias** del componente que no vi.
- **Qué hace un QA humano:** mantiene un checklist de regresión (página X, flujo Y, rol Z), lo corre antes de cada deploy a prod, detecta side-effects.

4. Experiencia de usuario — ¿esto se entiende?

- **Limitación IA:** Puedo escribir el código, pero no sé si un mantenedor sin experiencia entiende "PM02 Planificado" vs "PM01 Programado". No sé si el wizard de 3 pasos se siente largo. No sé si un color rojo en una celda se ve bien a las 3am en la pantalla del supervisor.
- **Ejemplo concreto esta semana:** Jorge pidió **5 veces** cambios de terminología ("Confianza IA" a "Reliability", "Cancel" a "Limpiar", quitar prefijo "WR-", etc). Cada uno porque el usuario final **no entendía** lo que estaba ahí. Una IA no detecta esto sin que alguien humano lo reporte.
- **Qué hace un QA humano:** valida con usuarios reales o pilotos, detecta fricción de UX, sugiere microcopy.

5. Confirmar que las features nuevas se usan correctamente en contexto

- **Limitación IA:** Agregué "5 pestañas clickables", "modal cancel con motivo", "nav bidireccional WR↔OT", "auto-close aviso al cerrar OT". **Cada una funciona aisladamente.** No puedo confirmar que el supervisor las usa en el orden correcto, que no se pisan entre ellas, que el flujo completo (crear aviso → validar → crear OT → ejecutar → cerrar → aviso auto-cierra) funciona **end-to-end sin que el usuario quede atascado**.
- **Qué hace un QA humano:** arma un "caso de uso completo" tipo Goldfields (falla real de chute → aviso P1 → OT fast track → ejecución en terreno → cierre con firma → métrica adherencia) y lo corre como si fuera un operador.

Costos reales sin QA humano

Esta semana Jorge reportó **al menos 8 bugs** que llegaron a él porque no había pasado por QA humano antes. Cada iteración con Jorge = 1-2 horas de ida-y-vuelta + frustración del stakeholder ("cuántas putas veces te tengo que decir"). Un QA humano habría cachado 6 de esos 8 antes.

Costo de oportunidad: cada bug que Jorge reporta consume ~1h de Jorge + 1h de David + pérdida de confianza. Un QA detecta el mismo bug en 15 min sin gastar el tiempo del cliente.

Propuesta de scope QA

Mínimo viable (20h/semana):

1. Smoke test diario post-deploy — 5 flujos críticos (login, crear aviso, aprobar, crear OT, cerrar).
2. Sesión de regresión semanal — checklist de 30+ pantallas antes de pasar a prod.
3. Caso de uso end-to-end — 1 por sprint, con datos cliente-like.
4. Reporte de bugs estructurado (pasos, screenshot, dato usado) que yo+Claude podamos resolver rápido.

Ideal (full-time):

- Todo lo anterior + escritura de casos automatizados (Playwright/Cypress) que el CI corre solo.
 - Validación con usuarios reales de Goldfields (piloto).
 - Métrica de bugs escapados a prod por mes.
-

Conclusión para José

Claude + yo cubrimos el 60% del QA. **Ese 40% restante son justo los bugs que el cliente ve** — UX, data real, regresión, flujos completos. Sin QA humano, cada semana que viene Jorge va a seguir reportando "el botón Cancel no funciona" y vamos a gastar sprints en cosas que un QA detecta en 30 min.

El QA no es lujo, es el puente entre "el código funciona" y "el producto funciona".